

Complexity Analysis

Algorithmic Performance Comparison: Stack-Based Evaluation vs. Expression Trees

Let n represent the total number of tokens in the postfix expression. This document analyzes both computational time bounds and memory footprints across evaluation models.

Evaluation Paradigms & Metrics

1. Stack-based Postfix Evaluation

Time Complexity $O(n)$

Each token in the postfix expression is processed exactly once in a single linear pass.

Space Complexity $O(n)$

The evaluation stack may contain up to n operands or intermediate results simultaneously.

2. Expression Tree Construction

Time Complexity $O(n)$

Each token is processed once during the syntax tree building phase.

Space Complexity $O(n)$

The resulting expression tree requires exactly n distinct nodes allocated in memory.

3. Expression Tree Evaluation

Time Complexity $O(n)$

Each node of the expression tree is visited precisely once during traversal.

Space Complexity $O(h)$

Where h represents the height of the expression tree (recursion stack depth).

Comparative Overview

Approach	Time Complexity	Space Complexity	Primary Memory Overhead
Stack Evaluation	$O(n)$	$O(n)$	Runtime evaluation stack
Tree Construction & Eval	$O(n)$	$O(n) + O(h)$	Complete structural tree nodes

Overall Summary

Both approaches achieve identical $O(n)$ time complexity. However, the expression tree approach requires additional structural memory because it stores the complete hierarchical representation of the expression rather than temporary scalar values.